MODULE I

Introduction to C Programming

C is a procedural programming language initially developed by Dennis Ritchie at Bell Laboratories in 1972. It was developed along with the UNIX operating system. The syntax of many programming languages including Java, PHP, JavaScript is primarily based on the C language.

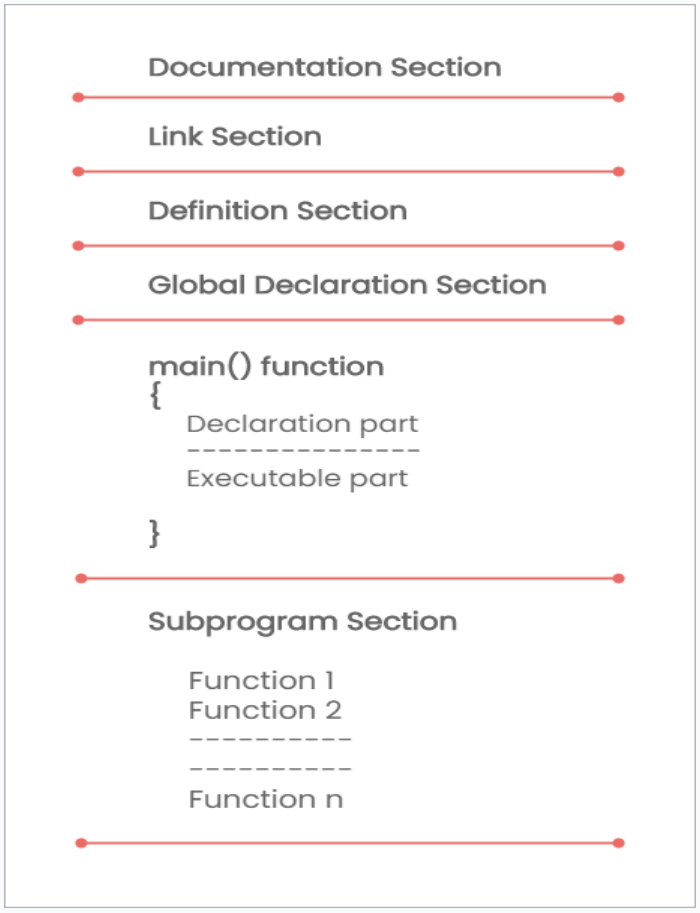
Features of C Programming Language

The growing popularity of C is most likely due to its many desirable characteristics. The following are the important features of the C programming language.

- Robust language – Well suitable for writing any complex programs.
- Fast and efficient programs – Because of variety of data types and operators.
- Rich set of inbuilt functions.
- Highly portable.
- Well suited for structured programming.
- Ability to extend itself.

Basic Structure of C Programs

A C program can be thought of as a collection of blocks known as functions. A function can be defined as a subroutine that contains one or more statements that perform a specific task. A C program may contain one or more sections as follows.



Structure of C Program

The **document section** of the program is made up of a set of comment lines containing details like the name of the program, author, etc.

The **link section** contains instructions to the compiler on how to link functions from the system library.

All symbolic constants are defined in the **definition section**.

Variables that are used in multiple functions are known as global variables. These variables are declared in the **global variables section**.

Every C program should have a **main()** function section. It is a special function used by the C to tell the computer where the program starts. The main() section is divided into two parts: the **declaration part** and the **executable part**. The declaration section declares all variables. The executable section contains at least one statement.

These two parts must be written inside the opening and closing braces. All statements in the declaration and executable sections are followed by a semicolon (;).

All user-defined functions that are called in the main function are written in the **subprogram section**.

Except for the **main** function section, all sections may be skipped when they are not required.

Hello World Program in C

The following program in C prints the message

“Hello World !” on the screen as below:

```
#include<stdio.h>

main()

{

printf("Hello World!");

}
```

Executing a C Program

Executing a program written in C involves the following steps:

1. Creating the program.
2. Compiling the program.
3. Linking the program with functions that are needed from the C library.
4. Executing the program.

ELEMENTS OF C LANGUAGE

Character Set

The set of alphabets, digits and special characters that are valid in the C language is called the **character set**.

Alphabets

Both lowercase (a – z) and uppercase (A – Z) alphabets are accepted by C.

Digits

C accepts all digits from 0 to 9.

Special Characters

C supports the usage of the following special characters.

,	–	$	]	&	^	|
<	(	:	#	{	!	–
>	)	%	?	}	*	\
.	;	[	‘	“	/	~
+						

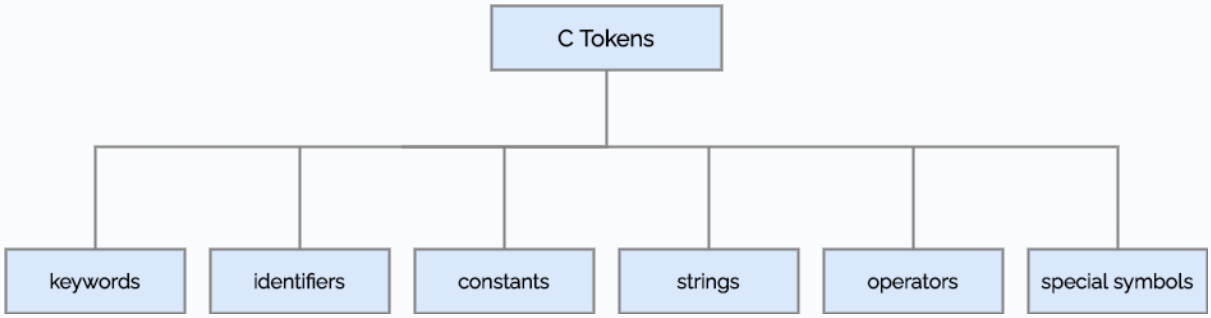
White spaces

Blank spaces, horizontal tab, carriage return, newline and form feed characters are allowed in C. White spaces may be used to

separate words, but they are not permitted between keywords and identifiers.

C tokens

The smallest individual units in a C program is known as C tokens. C has six types of tokens as shown in the figure.



Keywords

Predefined reserved words with fixed meanings are called keywords. Since keywords are the building blocks for program statements, they cannot be used as identifiers. Since C is case sensitive, all keywords must be written in lowercase. The list of all keywords available in C is given below:

- **auto**
- **break**
- **case**
- **char**
- const
- **continue**
- **default**
- **do**
- **double**
- **else**
- enum

- **extern**
- **for**
- **float**
- **if**
- **goto**
- **int**
- **long**
- **register**
- **return**
- **short**
- **static**
- sizeof
- signed
- struct
- **switch**
- typedef
- union
- **void**
- unsigned
- volatile
- **while**

Identifiers

The names of variables, functions, and arrays are referred to as identifiers. These are user-defined names consisting of a sequence of letters and digits. Identifiers must be unique.

The rules for naming identifiers in C

- Must consist of only letters, digits or underscore.
- Both uppercase and lowercase letters are permitted.
- The first character must be a letter of the alphabet or underscore.
- Only first 31 characters are significant.
- Cannot use a keyword.
- Must not contain white space.

For example:

```
int age;
float mark;
```

Here age and mark are identifiers.

Variables

A **variable** is a data name that can be used to store data. A variable may take different values at different times during the execution of a program. A variable name can be meaningfully chosen by the programmer, subject to the conditions listed below:

- Variable names can be made up of alphabets, digits, and the underscore character.
- They must begin with an alphabet. Some systems allow underscore as the initial character.
- Uppercase and lowercase letters are significant. That is, the variable Mark is not the same as mark or MARK.
- It should not be a keyword.
- White space is not allowed.

Some examples of valid variable names includes
Value, score, mark1, phone _number, Counter_1, etc.

Invalid examples include 123, (area), 22nd, char, etc.

Declaration of Variables

Declaration tells the compiler what the variable name is and what type of data the variable will hold. In C, variables can be declared using the following syntax:

```
datatype variable1;
```

Multiple variables can be declared by separating them with commas as follows.

```
datatype variable1, variable2, variable3;
```

For example,

```
int x;

float a, b, c;
```

Here

int and float are the keywords used to represent integer and real numbers respectively.

Initializing variables

Assigning value to a variable once declared is called initialization of a variable. In C, the variable declaration and initialization together takes the following form:

```
int x = 5;
```

The following ways are also acceptable.

```
int a, b=10, c = 5;

int x, y, z;

x = 10;

y = z = 5;
```

Here

b = 10, c = 5, x =10, y= 5, z = 5.

Constants

Fixed values that do not change throughout program execution are called **constants**. The keyword const can be used to define a constant as follows.

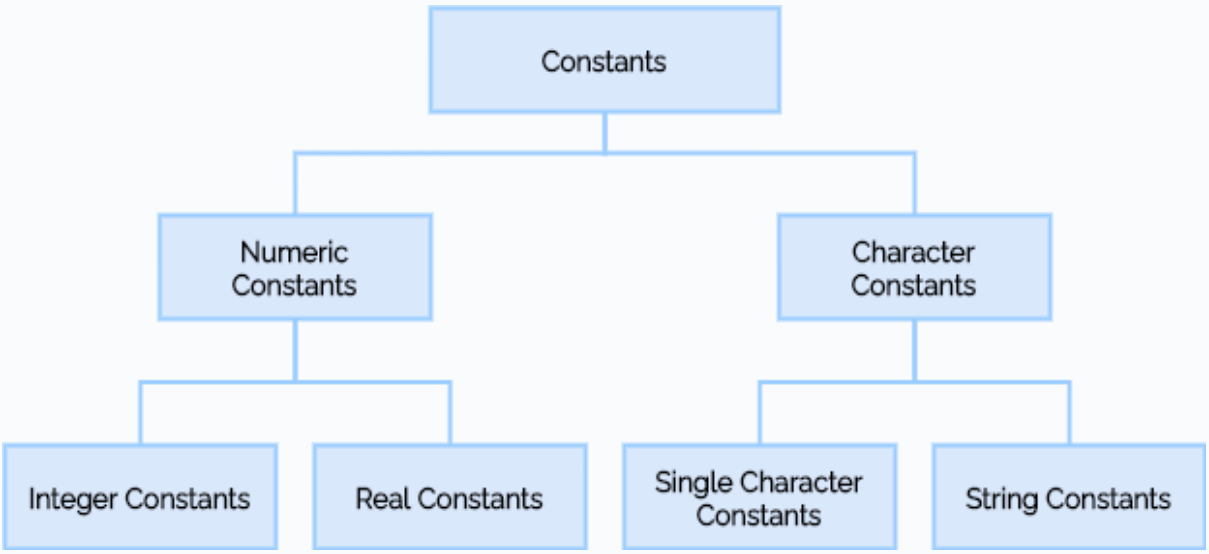
```
const datatype name = value;
```

For example

```
const double pi = 3.14;
```

will define a constant named 'pi' of type 'double' having the value '3.14'. The value of pi cannot be changed throughout the program.

Constants can also be defined using the #define preprocessor directive. You can learn about the symbolic constants in this tutorial.C supports the following types of constants.



Types of constants in C

Integer Constants

A sequence of digits is referred to as an integer constant. Spaces, commas, and non-digit characters are not permitted between digits. The integer constants are of three types.

- Decimal integer – Consist of a series of digits from **0** to **9**, followed by an optional – or + symbol.
- Octal integer – Made up of any combination of digits from **0** to **7**, with a leading 0.
- Hexadecimal integer – Series of digits preceded by **0x** or **0X**.

Example:
Decimal Integer: 120 -450 1 654321 +56

Octal Integer: 031 0 0321 04320

Hexadecimal: 0x2F 0X1A2 0xD10 0x

Real Constants

Numbers containing fractional parts are called real constants. Examples of real constants include 452.5 -.74 +321.2 etc.

Single Character Constants

A single character enclosed within a pair of single quote marks is called a **single character constant**. Examples of single character constants include 'F' 'c' '3'.

String Constants

A sequence of characters enclosed in double quotes is called a string constant. Letters, numerals, special characters, and blank spaces can all be used as characters. Some examples are "Hello" "Good Morning" "1996" "what?" etc.

Backslash Character Constants

C includes various special backslash character constants that can be used in output functions. They are known as escape sequences. The following table shows the list of backslash character constants in C.

Constant	Meaning
\a	Bell
\b	Back space
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\’	Single quote
\”	Double quote
\?	Question mark

Constant	Meaning
\\	Back slash
\0	Null

C Data Types

Data types are used to determine the size and type of data stored in a variable. For example,

```
int mark;
```

Here mark is a variable of type int. That is mark can store integer values.

C supports the following classes of data types:

- Primary data types.
- Derived data types.
- User-defined data types.

1.Primary Data Types

C supports five primary (fundamental) datatypes namely **integer**(**int**), **character**(char), **floating-point** (float), **double-precision floating-point**(double) and **void**(**void**). Many of them also include additional data types, such as long int and long double.

The following table contains various data types along with their size, range of values and format specifier.

Type	Size (bytes)	Range of values	Format specifier
int	2	-32,768 to 32767	%d
char	1	-128 to 127	%c
float	4	3.4E -38 to 3.4E +38	%f
double	8	1.7E -308 to 1.7E +308	%lf
short int	1	-128 to 127	%hd
unsigned int	2	0 to 65535	%u
unsigned short int	1	0 t0 256	%hu
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long double	10	3.4E -4932 to 1.1E +4932	%Lf

Integer

Integers are whole numbers having a range of values that are supported by a certain computer. In C, there are three types of integer storage: **short int**, **int**, and **long int**, which are available in both signed and unsigned forms. That is integers can store both zero, positive and negative values but no decimal values.

int roll;

int roll, mark;

```
short int id;
signed int x;
```

Short int represents relatively small integer values and takes half the storage space of a typical int number. unsigned int employ all of the bits for the magnitude of the number and are always positive. Because the default declaration implies a signed number, the use of the qualifier signed on integers is optional.

Floating point

Floating-point variables are used to store real numbers with 6 digits of precision. In C, floating-point numbers are defined by the keyword float.To define a number with 14 numbers of precision, the type double can be used. These are referred to as double-precision numbers. long double can be used to increase precision even further. Examples are

```
float rate;
double price;
```

Character

The char data type can be used to define a single character.

```
char demo = 'x';
```

Void

The **void** type has no values. This is typically used to specify the type of function that returns no value to the caller function.

2.User Defined Data Types

The data types that are defined by the user are called the user-defined derived data type.

STRUCTURE

A Structure is used to organize a group of related data items of different data types referring to a single entity. i.e., a single variable capable of holding data items of different data types.

The keyword used to create a structure is a struct. The advantage of using a structure is that the accessibility of members becomes easier since all the members of a specific structure get the allocation of continuous memory and therefore it minimizes the memory access time.

Generally, a structure can be declared as:

```
struct tag_name
{
  data_type1 svar_1;
  data_type2 svar_2;
  ....
  ....
  data_typen svar_n;
};
```

The structure variables can be defined

```
struct tag_name svar_1, svar_2 ...svar_n;
```

Example of a structure

```
struct sample {
  int a, b;
  float c, d;
  char e, f;
```

```
};
struct sample v1, v2, v3; //structure definition
```

UNION

A union is also a collection of different data types in **C** but that allows to store different data types in the same memory location. User can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple-purpose. A union is same as structures but the difference is that only one member can be accessed at a time because the memory is created only for one member which has the highest number of bytes in size.

A union is declared by using the keyword union and members of the union can be accessed by using dot (.) operator.

The declaration and definition of the union is:

```
union tag_name {
  data_type1 uvar_1;
  data_type2 uvar_2;
  ......
  data_typen uvar_n;
};
union tag_name uvar_1, uvar_2,....uvar_n;
```

Example

```
union sample {
  int age;
```

```
  float price;
  char name;
};
union sample s;
```

TYPEDEF

The keyword typedef is used to create a new name (alias) for an existing data type. It does not create a new data type. The syntax of using typedef is as follows:

```
typedef  existing_type  new_data_type;
```

ENUM

enum is a keyword used to create an enumerated data type. An enum (**enumerated data type**) is a special data type consisting of a set of named values called elements or members. It is mainly used to assign names to integral constants, which makes a program more readable.

The format for creating an enum type is

enum identifier (value1, value2, ... , valueN);

Example

```
enum Boolean {
    false,
    true
};
```

EMPTY DATA TYPE

void:

void keyword is an empty data type that represents **no value**. It is used in **functions** and **pointers**.

3.Derived Data Types

Derived types are data types that are derived from fundamental data types. Arrays, pointers, function types, and so on are examples.

ARRAY

Array in C stores multiple values of the same data type. That means we can have an array of integers, chars, floats, doubles, etc

Syntax

Type variable_name[size];

Example

int number[5];

float height[7];

POINTER

Pointer is a variable that can hold address of another variable

Syntax

type *var-name;

Example

```
int var = 20;

 int* p = &var;
```

Here, 20 is assigned to the var variable. And, the address of var is assigned to the p pointer.

FUNCTION

A functionn is a self contained block of code that performs a particular task.

Symbolic Constants

A symbolic constant is a name given to any constant. In C, the pre-processor directive #define is used for defining symbolic constants. #defineinstructions are usually placed at the beginning of the program. By convention, the names of symbolic constants are written in uppercase, but this is not compulsory. The syntax for creating a symbolic constant is as follows:

```
#define constant_name value
```

For example:

```
#define PI 3.14
```

It will define a symbolic constant PI having value 3.14. When we use PI in our program, it will be replaced with 3.14 by the compiler automatically.

The rules below apply to a #define statement that defines a symbolic constant.

- No blank space is allowed between the # symbol and the word define.
- The character in the line should be #.
- A blank space is required between the constant name and #define and between the constant name and the value.
- A semicolon must not be used at the end of a #define statement.

- Within the program, the symbolic constant should not be assigned any other value.

Example

```
#include <stdio.h>

#define PI 3.1415

int main()

{

float radius=10, area;

area = PI*radius*radius;

printf("Area=%.2f",area);

return 0;

}
```

MODULE 2

C OPERATORS

An **operator** is a symbol that instructs a computer to perform certain operations. Operators are typically used as part of mathematical or logical expressions

Arithmetic Operators

An arithmetic operator performs mathematical operators such as addition, subtraction etc. C supports all basic arithmetic operators are

Operator	Meaning	Example
+	Addition or Unary plus	8 + 2 = 10
–	Subtraction or Unary minus	8 – 2 =6
*	Multiplication	8 * 2 = 16
/	Division	8 / 2 = 4
%	Modulo division	13 % 2 = 1

Relational Operators

The relationship operators are used to check the relationship between two operands. The value of a relational operator is either 1 (true) or 0 (false).

Operator	Meaning	Example
<	is less than	a < b is true.
<=	is less than or equal to	a <= b is true
>	is greater	a > b is false
>=	is grater than an equal to	a >= b is false
==	is equal to	a == b is false

Operator	Meaning	Example
!-	is not equal to	a != b is true

Logical Operators

The logical operators are used to make a decision by testing one or more conditions.

- && – logical AND – True only if all operands are true.
- || – logical OR – True only if either one operand is true.
- ! – logical NOT – True only if the operand is 0

Assignment Operator

Assignment operators are used to assigning values to a variable. The assignment operator in c is =.

Operator	Example	Same as
+=	a += b	a = a + b
-=	a -= b	a = a – b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

Increment and Decrement Operator

In C, the operators ++and --are called the increment and decrement operators. The increment operator ++increases the value of

an operand by 1 and the decrement operator --decreases the value of an operand by 1. They both are unary operators.

Bitwise Operator

For manipulation of data at bit level, C supports special operators known as bitwise operators.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
`	Bitwise complement
<<	Shift left
>>	Shift right

Conditional Operator

C has a special ternary operator ?: called conditional operator. The conditional operator behaves similarly to the 'if-else' statement. The syntax

```
condition ? true-statement : false-statement;
```

Here expression1 is a boolean condition that can be either true or false. The conditional operator will return expression2, if the value of expression1 is true. Similarly, if the value of expression1 is false, then the operator will return expression3

```
max = (a > b) ? a : b ;
```

First, the expression a > b is evaluated. If it is true, the statement will be equivalent to max = a;. If a > b is false, then the statement will be equivalent to max = b.

Special Operators

C also support some special operators such as comma operator, sizeof operator, pointer operators, etc.

The Comma Operator

The comma operator is used to link the related expression together. For example: **float** a, b, c;

The sizeof Operator

The sizeof operator return the number of bytes a operand occupies. It can be used to know the size of variables, constants, arrays, structures etc. For example:

```
x = sizeof(mark);
```

Arithmetic Expressions

An arithmetic expression is a combination of variables, constants, and arithmetic operators. Arithmetic operators are

+, -, *, / and %. The operands include integer and floating-type numbers. Some algebraic expressions are

Algebraic Expression	C Expression
(a+b)(a-b)	(a + b) * (a-b)
(ab)/c	(a * b) / c

Arithmetic expressions are evaluated using an assignment statement of the form variable = expression. The expression is evaluated first and the value is assigned to the variable. An example of an evaluation statement is,

c = a - b / d + e

Precedence of Arithmetic Operators in C

To determine the meaning and value of an expression in an unambiguous manner, we apply the operator precedence and associativity rules. Arithmetic expressions without parentheses are evaluated from left to right using the rules of operator precedence. In C, arithmetic operators have two different priority levels.

- Hight priority * / %
- Low priority + -

The basic procedure for evaluating an expression includes two passes from left to right. The high priority operators are applied during the first pass and the low priority operators are applied in the second pass.

For example, the statement x = 8 – 25 / 5 + 2 * 5 – 7 is evaluated as follows,

- **First pass**
  - x = 8 - 5 + 2 * 5 -7
  - x = 8 - 5 + 10 - 7
- **Second pass**
  - x = 3 + 10 - 7
  - x = 13 - 7
  - x = 6

These steps are illustrated in the following figure.

However, parentheses can be used to change the order in which an expression is evaluated. The expression within parentheses assumes the highest priority. The following rules are used for evaluating expressions containing parentheses.

- Parenthesized subexpressions are evaluated from left to right.
- When there are nested parentheses, the evaluation starts with the innermost sub-expression.
- The order of application of operators in evaluating sub-expressions is determined using the precedence rule.
- When two or more operators with the same precedence level exist in a sub-expression, the associativity rule is applied.
- Arithmetic expressions are evaluated from left to right using the rules of precedence.
- The expressions within parentheses assume the highest priority.

Operator Precedence and Associativity in C

The following table shows the complete list of C operators, their precedence levels, and their rules of association.

Operator	Description	Associativity	Rank
()	Function call	Left to right	1

Operator	Description	Associativity	Rank
[]	Array element reference		
+	Unary plus	Right to left	2
–	Unary minus		
++	Increment		
—	Decrement		
!	Logical negation		
~	Ones complement		
*	Pointer reference		
&	Address		
sizeof	Size of an object		
(type)	Type cast (conversion)		
*	Multiplication	Left to right	3

Operator	Description	Associativity	Rank
/	Division		
%	Modulus		
+	Addition	Left to right	4
–	Subtraction		
<<	Left shift	Left to right	5
>>	Right shift		
<	Less than	Left to right	6
<=	Less than or equal to		
>	Greater than		
>=	Greater than or equal to		
==	Equality	Left to right	7
!=	Inequality		

Operator	Description	Associativity	Rank
&	Bitwise AND	Left to right	8
^	Bitwise XOR	Left to right	9
|	Bitwise OR	Left to right	10
&&	Logical AND	Left to right	11
||	Logical OR	Left to right	12
?:	Conditional operator	Right to left	13
=	Assignment operator	Right to left	14
,	Comma operator	Left to right	15

Mathematical Functions

Mathematical functions such as sqrt, cos, sin, log, etc. these functions are supported by the header file is <math.h>

Library Functions

Library functions are built-in functions that are grouped together and placed in a common location called library. All C standard library functions are declared by using many header files. These library functions are created at the time of designing the compilers.